C PROGRAMMING — ASSIGNMENT 1

Task 1: Understand the Problem and Identify the Data

Marks: 4 | Total assignment: 50 marks

Analysis

The program receives a household number, customer category, previous meter reading and current meter reading. It validates the readings, computes consumed units, applies the tariff slabs, adds the fixed charge, and prints a bill with two decimal places.

Inputs: household_id (identifier; string is suitable if leading zeroes are possible), category (enumerated/text category), previous_reading and current_reading (meter register values from 0 through 65535).

Computed values: units_consumed; energy_charge_paise; fixed_charge_paise; total_bill_paise. Tariff rates and slab limits do not change during one run and should be named constants.

Assumptions: the register is a 16-bit unsigned counter; a current reading lower than the previous reading means one rollover unless the input is otherwise declared invalid; at most one rollover occurs between readings; the supplied tariff is 0 paise for units 1–100, 225 paise for units 101–200, and 450 paise above 200; fixed charge is 5000 paise. A bill must never be negative and total must equal fixed charge plus energy charge.

Types should reflect their meaning: meter readings are non-negative; units and household count can be large; monetary amounts are stored as integer paise to avoid binary floating-point rounding. Negative user input is rejected before conversion to the meter type.

Data and justification

Data item	Purpose	Recommended representation
Household ID	Identifies a customer	char array/string
Category	Tariff/customer category	enum or validated string
Previous/current reading	Meter register	uint16_t, or unsigned short when its size is verified
Units consumed	Difference including rollover	uint32_t or unsigned int
Slab limits/rates	Tariff rules	named macros or const values
Energy charge	Cost of consumed units	int64_t/long long paise
Fixed charge	Fixed monthly amount	int64_t/long long paise
Total bill	Sum of charges	int64_t/long long paise
District total	Sum of many bills	int64_t/long long paise with overflow checks where needed

Text versus integer representation

If each reading is kept as a string such as "01530", comparison and arithmetic require parsing first. For d digits, parsing and string comparison take O(d) time and each reading occupies O(d) characters plus a terminator. A fixed-width integer uses constant space and arithmetic/comparison O(1). Text can preserve leading zeroes and is useful for raw input, but the validated numeric value should be converted before billing.

Pseudocode

READ household ID, category, previous reading, current reading

VALIDATE both readings are in [0, 65535]

IF current >= previous: units = current - previous

ELSE: units = (65535 - previous) + 1 + current

energy = slab_charge(units)

total = fixed_charge + energy

ASSERT total >= 0 and total == fixed_charge + energy

PRINT bill

C Source Code / Implementation

```
/* Task 1: basic billing model and invariants */
#include <stdio.h>
#include <stdint.h>
```

```c
#include <inttypes.h>
#define FIXED_CHARGE_PAISE INT64_C(5000)

static uint32_t consumption(uint16_t previous, uint16_t current) {
    return (uint16_t)(current - previous);
}
int main(void) {
    const char household_id[] = "H001";
    const char category[] = "Domestic";
    uint16_t previous = 1200, current = 1530;
    uint32_t units = consumption(previous, current);
    int64_t energy = 0; /* first 100 units are free in this example */
    int64_t total = FIXED_CHARGE_PAISE + energy;
    printf("Household: %s, category: %s\n", household_id, category);
    printf("Units: %" PRIu32 "\nEnergy: %" PRId64 " paise\n", units, energy);
    printf("Fixed: %" PRId64 " paise\nTotal: %" PRId64 " paise\n",
           FIXED_CHARGE_PAISE, total);
    return (total >= 0 && total == FIXED_CHARGE_PAISE + energy) ? 0 : 1;
}
```

Submission note

Compile and run the code on your own system, save actual terminal output/screenshots, and include them in your report. Review the code and be prepared to explain it during the viva. Any machine-specific measurements must come from your own execution.